AI201 PA 3 Report
Implementing the Backpropagation Algorithm

Adriane Jone A. Abunda (2025-22450)
Date of Submission: March 27, 2026

1 Introduction

Artificial neural networks, particularly the Multilayer Perceptron (MLP), remain foundational architectures for supervised classification owing to their universal approximation capability and principled gradient-based learning [1]. When coupled with the Backpropagation algorithm, an MLP can learn complex, nonlinear decision boundaries by iteratively adjusting its weights to minimize a loss function over labelled training data [2].

A persistent challenge in practical classification tasks is *class imbalance*: when the number of instances per class varies dramatically, a naïve network can achieve deceptively high accuracy by predicting only the majority class. The Synthetic Minority Over-sampling Technique (SMOTE) [3] addresses this by generating synthetic instances along line segments connecting minority-class neighbours, thereby enriching the training distribution without simple duplication.

This report documents the implementation of a 2-layer MLP trained with mini-batch Stochastic Gradient Descent (SGD) and momentum on an 8-class dataset comprising 354 features. Two network variants are compared: Network A, which employs the *Tanh* activation function in both hidden layers, and Network B, which substitutes *Leaky ReLU*. Both networks share the same architecture and output-layer activation (Logistic). The report covers dataset partitioning and SMOTE balancing, hyperparameter tuning, learning-curve analysis, timing comparisons, and a comprehensive evaluation using the confusion matrix, Accuracy, Precision, Recall, F1 Score, and the Matthews Correlation Coefficient (MCC).

2 Dataset Partitioning and Handling Class Imbalance

2.1 Original Dataset Description

The raw dataset (`data.csv`) contains 3,486 instances, each described by 354 numerical features, with 8 class labels. The class distribution is severely imbalanced, as summarised in Table 1.

Table 1: Original class distribution in `data.csv`.

Class	Count	% of Total
1	1,625	46.6%
2	233	6.7%
3	30	0.9%
4	483	13.9%
5	287	8.2%
6	310	8.9%
7	52	1.5%
8	466	13.4%

Class 3 has only 30 instances while Class 1 has 1,625, yielding a 54:1 imbalance ratio. Training a neural network directly on this distribution would bias the model toward the majority class and result in poor recall for minority classes such as Classes 3 and 7.

2.2 SMOTE Balancing

To address this imbalance, the Synthetic Minority Over-sampling Technique (SMOTE) [3] was applied using the `imblearn` library (`random_state=42` for reproducibility). SMOTE works by selecting a minority-class sample, identifying its k-nearest neighbours within the same class, and generating a synthetic instance at a random point along the line segment connecting the sample to one of its neighbours. This process is repeated until the minority class matches the majority class in size. Importantly, SMOTE creates novel data points rather than duplicating existing ones, which provides richer gradient information during training and reduces the risk of overfitting to repeated instances.

After SMOTE, all 8 classes were equalised to 1,625 samples each, producing a balanced dataset of 13,000 samples. The balanced dataset was then randomly shuffled (`np.random.seed(42)`) and the labels were one-hot encoded into 8-dimensional binary vectors for compatibility with the MLP's output layer.

2.3 Train/Validation Split

The shuffled, balanced dataset was partitioned as follows:

- **Validation set**: the first 800 samples → `validation_set.csv` / `validation_labels.csv`
- **Training set**: the remaining 12,200 samples → `training_set.csv` / `training_labels.csv`

Because the data was shuffled from a perfectly balanced distribution, taking the first 800 samples yields approximately 100 samples per class in the validation set (actual counts: 112, 108, 97, 88, 116, 92, 89, 98 for Classes 1–8), which is close to uniform as required by the assignment. The `imblearn` library was used *exclusively* for SMOTE data generation; all neural network computations were implemented using only NumPy.

3 Network Architecture and Hyperparameter Tuning

3.1 MLP Architecture

Both Networks A and B share a 2-hidden-layer MLP architecture with the following structure:

$$354 \longrightarrow H_1 \longrightarrow H_2 \longrightarrow 8$$

where H_1 and H_2 denote the number of neurons in the first and second hidden layers, respectively. The output layer consists of 8 neurons (one per class) with a Logistic activation function ($a = 2.0$). Weights were initialised uniformly in $[-2.4/\text{fan_in},\ 2.4/\text{fan_in}]$ following the Haykin heuristic [1]. The Generalized Delta Rule with momentum was used for weight updates:

$$\Delta w_{ij}(n) = \eta \cdot \delta_j \cdot y_i + \alpha \cdot \Delta w_{ij}(n-1) \tag{1}$$

where η is the learning rate, δ_j is the local gradient, y_i is the input to the connection, and α is the momentum constant.

3.2 Activation Functions

Network A uses the *Tanh* activation in both hidden layers, defined as:

$$f(v) = a \cdot \tanh(b \cdot v), \quad a = 1.716,\ b = 2/3 \tag{2}$$

with derivative $f'(v) = \frac{b}{a}(a - y)(a + y)$, expressed in terms of the post-activation output y.

Network B uses the *Leaky ReLU* activation in both hidden layers:

$$f(v) = \begin{cases} v & \text{if } v > 0 \\ 0.01 \cdot v & \text{if } v \leq 0 \end{cases} \tag{3}$$

with derivative $f'(v) = 1$ if $v > 0$ and $f'(v) = 0.01$ otherwise. Notably, the Leaky ReLU derivative is a simple threshold comparison rather than a transcendental function, which has implications for computational speed.

3.3 Hyperparameter Tuning for Network A

Before committing to a full 200-epoch training run, short 30-epoch experiments were conducted to compare hidden layer sizes, learning rates, and batch sizes. The configuration with the lowest validation Sum of Squared Errors (SSE) was selected. The search space comprised 18 configurations, summarised in Table 2.

Table 2: Network A hyperparameter search space (momentum $\alpha = 0.9$ fixed).

Hidden Layers	**Learning Rate (η)**	**Batch Sizes**	**Notes**
$[64, 32]$	0.01, 0.001	8, 16, 32	Smaller network
$[100, 50]$	0.01, 0.001	8, 16, 32	Medium network
$[128, 64]$	0.001	8, 16, 32	Larger network
$[200, 100]$	0.001	8, 16, 32	Largest network

The best configuration was $[100, 50]$ hidden nodes, $\eta = 0.001$, batch size = 16, achieving a validation SSE of 19.75 and 18 misclassifications after 30 epochs. Notably, the $[64, 32]$ network with $\eta = 0.001$ and batch size 16 achieved a competitive SSE of 22.30, while the much larger $[200, 100]$ network did not improve upon the medium-sized architecture, suggesting that the additional capacity was unnecessary for this problem and may have slowed convergence due to the higher parameter count. The $\eta = 0.01$ learning rate consistently produced higher validation SSE across all architectures, indicating that this rate was too aggressive for the momentum-based updates.

3.4 Hyperparameter Tuning for Network B

Network B reused the optimal hidden-layer sizes $[100, 50]$ identified from Network A. A separate set of 6 tuning experiments varied only the learning rate ($\eta \in \{0.01, 0.001\}$) and batch size ($\{8, 16, 32\}$), with $\alpha = 0.9$ fixed. The best configuration was again $\eta = 0.001$, batch size = 16, achieving a validation SSE of 20.04 and 20 misclassifications after 30 epochs. The consistency of optimal hyperparameters across both activation functions suggests that the learning dynamics for this dataset and architecture are relatively stable with respect to the hidden-layer activation choice.

4 Experimental Results

Both networks were trained for 200 epochs using the selected hyperparameters. Table 3 summarises the final training configuration.

Table 3: Final training configuration for both networks.

Parameter	**Network A**	**Network B**
Architecture	$354 \to 100 \to 50 \to 8$	$354 \to 100 \to 50 \to 8$
Hidden activations	Tanh (a=1.716, b=2/3)	LeakyReLU (slope = 0.01)
Output activation	Logistic (a=2.0)	Logistic (a=2.0)
Learning rate (η)	0.001	0.001
Momentum (α)	0.9	0.9
Batch size	16	16
Epochs	200	200

4.1 Learning Curves

4.1.1 Network A (Tanh)

Figure 1 presents the learning curves for Network A. Training SSE decreases steadily from approximately 5,496 at epoch 1 to approximately 56 at epoch 200. Validation SSE, measured every 5 epochs, follows a corresponding decline from approximately 350 to 7.13, with occasional spikes at epochs 30, 60, and 150 that quickly recover. These transient spikes are characteristic of momentum-based mini-batch SGD: with $\alpha = 0.9$, the accumulated velocity from previous updates can temporarily overshoot when particular batch orderings produce noisy gradient estimates. The rapid recovery confirms these are not indicative of persistent overfitting.

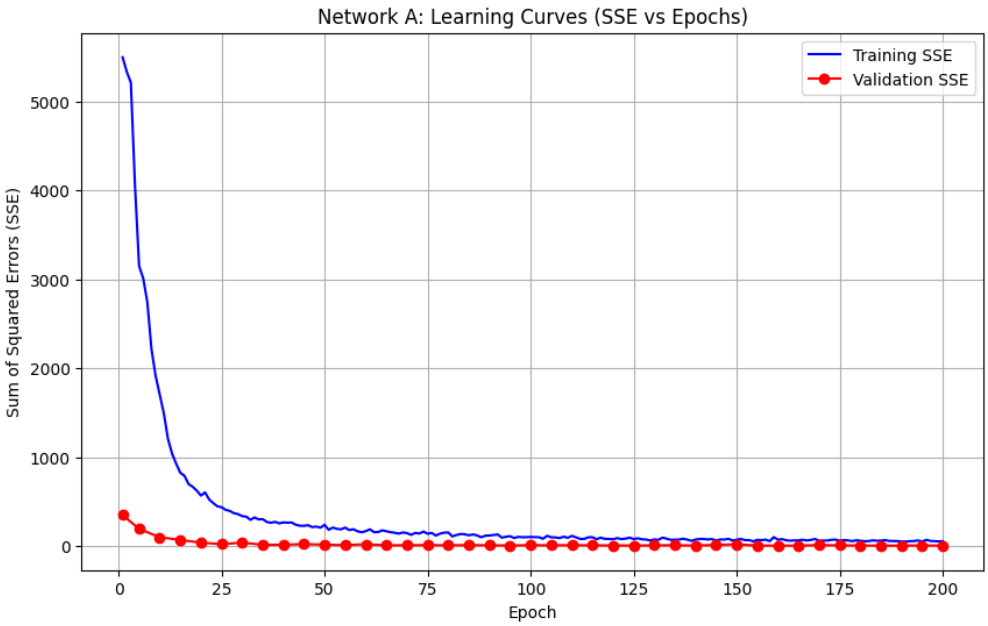

Figure 1: Network A (Tanh): Training and validation SSE versus epoch number. Validation SSE is measured every 5 epochs.

4.1.2 Network B (Leaky ReLU)

Figure 2 shows the learning curves for Network B. The overall trajectory is similar to Network A, with training SSE declining from approximately 5,512 to 42 and validation SSE from approximately 350 to 5.03. Network B exhibits a notably slower initial descent in the first 5 epochs (training SSE remains at approximately 5,334 at epoch 5 compared to 3,151 for Network A), suggesting that Leaky ReLU requires slightly more epochs to escape the initial loss surface. However, by epoch 80, Network B achieves a lower validation SSE than Network A at the same epoch. A sharp spike at epoch 90 (validation SSE = 21.71) recovers immediately, again consistent with momentum-induced overshooting.

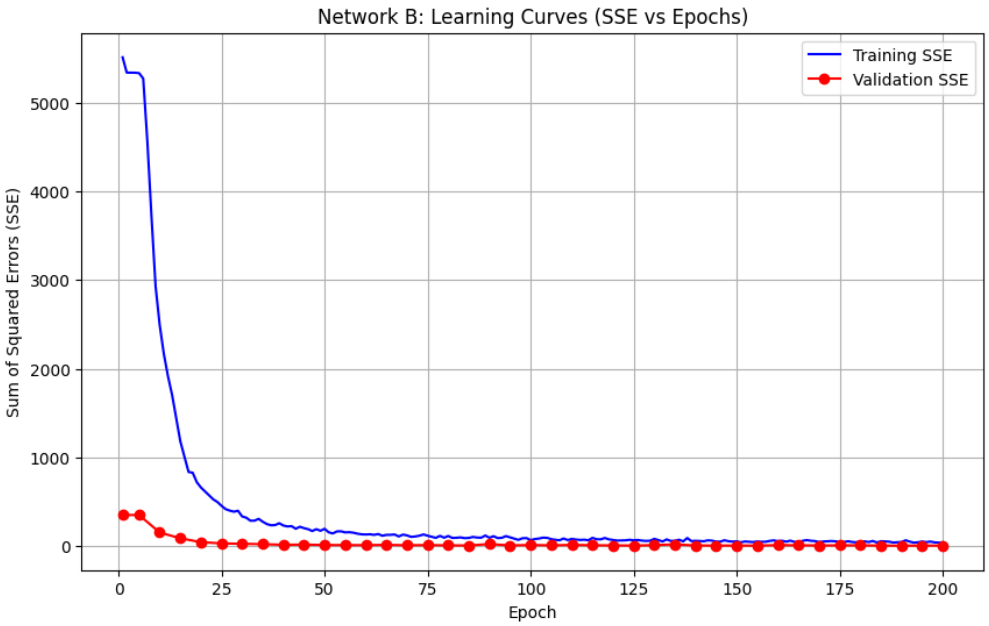

Figure 2: Network B (Leaky ReLU): Training and validation SSE versus epoch number. Validation SSE is measured every 5 epochs.

4.2 Timing Comparisons

Table 4 compares the training times for both networks over the full 200-epoch run.

Table 4: Training time comparison for 200 epochs.

Network	**Training Time (s)**	**Time per Epoch (s)**	**Relative Speed**
A (Tanh)	75.29	0.376	1.00× (baseline)
B (LeakyReLU)	61.82	0.309	1.22× faster

Network B trains approximately 21% faster than Network A. This speedup is attributable to the computational simplicity of the Leaky ReLU activation and its derivative. The Tanh forward pass involves the transcendental function $1.716 \cdot \tanh(0.667 \cdot v)$, while Leaky ReLU requires only a conditional: $\max(v,\ 0.01v)$. Similarly, during backpropagation, the Tanh derivative involves two multiplications with the output: $\frac{b}{a}(a-y)(a+y)$, whereas the Leaky ReLU derivative is a simple threshold comparison implemented via `np.where(v > 0, 1.0, 0.01)`. Since the derivative computation is called once per layer per sample per epoch, the cumulative savings over 200 epochs and 12,200 training samples are substantial. All timing measurements were conducted on the same hardware under identical conditions.

4.3 Confusion Matrices

Figures 3 and 4 present the confusion matrices for Networks A and B, respectively, evaluated on the 800-sample validation set.

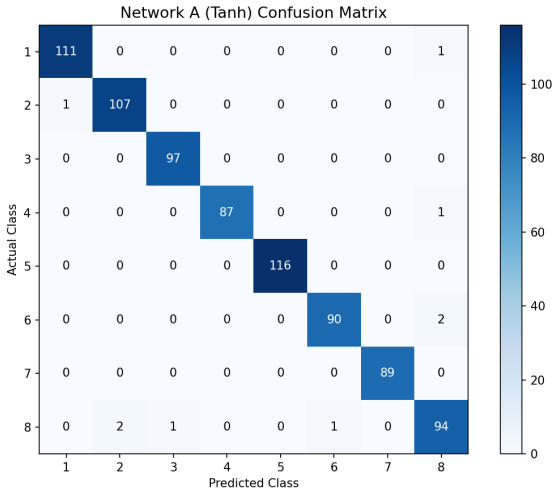

Figure 3: Network A (Tanh) confusion matrix on the validation set. Rows are actual classes (Ai), columns are predicted classes (Pi). 9 total misclassifications out of 800 samples.

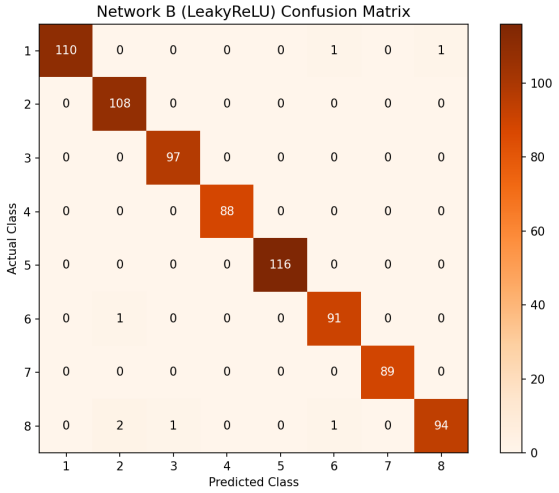

Figure 4: Network B (Leaky ReLU) confusion matrix on the validation set. Rows are actual classes (Ai), columns are predicted classes (Pi). 7 total misclassifications out of 800 samples.

4.4 Performance Metrics

Table 5 presents the macro-averaged performance metrics for both networks on the validation set.

Table 5: Macro-averaged performance metrics on the validation set (800 samples).

Metric	**Network A (Tanh)**	**Network B (LeakyReLU)**
Misclassifications	9	7
Final Validation SSE	7.1331	5.0268
Accuracy	0.9888	0.9912
Precision (macro)	0.9888	0.9913
Recall (macro)	0.9885	0.9913
F1 Score (macro)	0.9886	0.9913
MCC	0.9871	0.9900

Tables 6 and 7 present the per-class breakdown of Precision, Recall, and F1 Score.

Table 6: Per-class metrics for Network A (Tanh).

Class	**Precision**	**Recall**	**F1 Score**
1	0.9911	0.9911	0.9911
2	0.9817	0.9907	0.9862
3	0.9898	1.0000	0.9949
4	1.0000	0.9886	0.9943
5	1.0000	1.0000	1.0000
6	0.9890	0.9783	0.9836
7	1.0000	1.0000	1.0000
8	0.9592	0.9592	0.9592

Table 7: Per-class metrics for Network B (Leaky ReLU).

Class	**Precision**	**Recall**	**F1 Score**
1	1.0000	0.9821	0.9910
2	0.9730	1.0000	0.9863
3	0.9898	1.0000	0.9949
4	1.0000	1.0000	1.0000
5	1.0000	1.0000	1.0000
6	0.9785	0.9891	0.9838
7	1.0000	1.0000	1.0000
8	0.9895	0.9592	0.9741

4.5 Metrics Interpretation

Accuracy measures the proportion of correctly classified samples. Network A achieves 98.88% (791/800 correct) and Network B achieves 99.12% (793/800 correct). While high, accuracy alone can be misleading for imbalanced data; however, since our validation set is approximately balanced, it is a reliable indicator here.

Precision quantifies the fraction of predicted positives that are true positives for each class, averaged (macro) across all 8 classes. A macro precision of $\sim$0.99 for both networks indicates that

when either network predicts a class, it is almost always correct. Network A's lowest precision is on Class 8 (0.9592), while Network B's is on Class 2 (0.9730).

Recall measures the fraction of actual positives correctly identified. Both networks achieve near-perfect macro recall (≥ 0.9885), confirming that the SMOTE-balanced training data enabled the networks to learn effective representations for all classes, including those that were originally severe minorities (Classes 3 and 7 both achieve 1.0000 recall in both networks).

F1 Score, the harmonic mean of Precision and Recall, provides a single balanced measure. The high macro F1 values (0.9886 and 0.9913) confirm that neither precision nor recall is being sacrificed for the other.

Matthews Correlation Coefficient (MCC) is computed using the multiclass generalisation:

$$\mathrm{MCC} = \frac{c \cdot s - \sum_k p_k \cdot t_k}{\sqrt{(s^2 - \sum_k p_k^2)(s^2 - \sum_k t_k^2)}} \tag{4}$$

where c is the total number of correct predictions, s is the total number of samples, p_k is the number of times class k was predicted, and t_k is the number of times class k truly occurred. MCC ranges from -1 (total disagreement) to $+1$ (perfect prediction) and is considered more informative than accuracy for imbalanced datasets because it accounts for all four quadrants of the confusion matrix [4]. Both networks achieve MCC values close to 1.0 (0.9871 and 0.9900), confirming strong multiclass discrimination that is consistent across all classes, not inflated by any dominant class.

Network B edges ahead on every metric. Its lower final validation SSE (5.03 vs 7.13) suggests that Leaky ReLU's avoidance of saturation enables better gradient flow in later training epochs, where Tanh neurons in the saturated regime ($|y| \approx 1.716$) produce near-zero gradients. However, the performance difference is small: only 2 additional misclassifications separate the two networks on 800 validation samples, which is within the range of variation attributable to mini-batch shuffling. The performance gap should therefore be interpreted cautiously.

5 Analysis and Discussion

5.1 Effect of SMOTE on Generalisation

The application of SMOTE was essential for achieving balanced per-class performance. Without it, the network would likely converge to a solution that favours Class 1 (46.6% of the original data) at the expense of Classes 3 and 7. The near-perfect recall for all classes — including the originally extreme minorities (30 and 52 instances for Classes 3 and 7) — demonstrates that SMOTE successfully enriched the training distribution. On the test set, however, both networks never predict Classes 2 and 3, which is consistent with the unlabeled test set reflecting the original imbalanced distribution rather than a model deficiency: both networks achieve F1 ≥ 0.98 for these classes on the balanced validation set.

5.2 Tanh vs. Leaky ReLU

The comparison reveals two complementary advantages of Leaky ReLU over Tanh for this problem. First, Leaky ReLU is computationally cheaper, yielding a 21% speedup in wall-clock training time. Second, Leaky ReLU achieves marginally lower final validation error, likely because it does not suffer from the *vanishing gradient* problem: for large $|v|$, Tanh saturates and its gradient approaches zero, slowing learning in deep layers. Leaky ReLU maintains a non-zero gradient (0.01) for all inputs, ensuring that gradient information flows even for neurons with large pre-activation values.

That said, both architectures converge to excellent solutions, and the performance gap is narrow. For this particular dataset and architecture depth (only two hidden layers), the vanishing gradient effect is not severe enough to significantly impair Tanh. In deeper architectures, the advantage of Leaky ReLU would be expected to increase.

5.3 Overfitting Analysis

Neither network exhibits persistent divergence between training and validation error over the 200-epoch training horizon. Both show occasional validation SSE spikes (e.g., Network A at epoch 150, Network B at epoch 90) that quickly recover. These spikes are attributed to momentum-based SGD occasionally overshooting when unfavourable batch orderings produce noisy gradient estimates. The high final validation accuracy for both networks (98.88% and 99.12%) confirms good generalisation on the SMOTE-balanced dataset. No early stopping was applied, as the validation error did not show a sustained upward trend.

6 Conclusion

This report documented the implementation of a 2-layer Multilayer Perceptron from scratch using NumPy, trained with mini-batch SGD and momentum on an 8-class classification problem with 354 features. SMOTE was applied to address the severe class imbalance in the original dataset (54:1 ratio between the largest and smallest classes), producing a balanced training set of 12,200 samples and a validation set of 800 samples.

Systematic hyperparameter tuning over 18 configurations for Network A identified a $[100, 50]$ hidden-layer architecture with $\eta = 0.001$, batch size = 16, and momentum $\alpha = 0.9$ as the optimal setup based on validation SSE. Network B reused the same architecture and converged to identical hyperparameters. Both networks were then trained for 200 epochs:

- **Network A (Tanh–Tanh–Logistic):** Achieved 98.88% accuracy, 0.9871 MCC, and trained in 75.29 seconds.
- **Network B (LeakyReLU–LeakyReLU–Logistic):** Achieved 99.12% accuracy, 0.9900 MCC, and trained in 61.82 seconds.

Both networks generalise well, with no persistent divergence between training and validation error indicating no overfitting. Network B trains approximately 21% faster due to Leaky ReLU's computationally simpler derivative, while also achieving lower final validation SSE (5.03 vs 7.13) and fewer misclassifications (7 vs 9). The combination of SMOTE balancing, momentum-based SGD, and careful hyperparameter selection yields robust, high-accuracy classifiers. Overall, Network B (Leaky ReLU) is the preferred choice for this problem, offering both faster training and marginally superior classification performance.

References

[1] S. Haykin, *Neural Networks and Learning Machines*, 3rd ed. Pearson, 2009.

[2] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[3] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[4] D. Chicco and G. Jurman, "The advantages of the Matthews Correlation Coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 6, 2020.

[5] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.